AUTONATEAI SCREEPS LEAGUE

Scout Before Contact

A local Screeps competition module

docs/competitions/03-scout-before-contact.md

Summary

The learner must scout nearby rooms, classify threats, and decide where to expand or avoid before spawning combat creeps.

Skill Target

- Primary: intel collection and room classification.
- Secondary: Memory hygiene, route planning, threat vocabulary.
- Program track: Lead Developer, Episode 11.
- Recommended episode: `docs/tutorials/11-scouting.md`.

Prerequisites

- Learner can spawn and route a scout.
- Local server CLI is available.
- A bot can be spawned a few rooms away.

Setup

Open the CLI:

```
docker exec -it autonate-screeps screeps-launcher cli
```

Spawn one rival bot in an unowned room a few rooms away:

```
bots.spawn('tooangel', 'W3N1', { cpu: 100, gcl: 1 })
```

Challenge

Before attacking or expanding, create a scout that records owner, reservation, hostile creeps, structures, sources, and last-seen time for at least five rooms.

Pressure

A persistent bot colony creates an evolving map state.

Win Condition

The learner passes when:

- `Memory.rooms` has fresh intel for five rooms.
- The bot room is correctly marked as owned or hostile.
- The learner chooses one safe expansion candidate and one avoid candidate.
- The decision references recorded data, not only visual inspection.

Scoring

Metric	Full Credit	Partial Credit
Outcome	Five-room intel map with actionable classification.	Fewer rooms or missing threat fields.
Code behavior	Scout updates stale intel and avoids useless repeats.	Scout records data but route behavior is manual.
Debugging process	Learner explains decision using Memory evidence.	Learner gives a visual-only explanation.

Replay

Remove or respawn the bot in a different room, clear stale intel, then rerun the scout.

```
bots.removeUser('BOT_USERNAME')
```

Reflection

- What made a room unsafe?
- What data was missing from your first scout version?
- How old can intel be before you stop trusting it?

Extensions

- Easier: scout three rooms.
- Harder: require automatic route selection from exits.
- Team variant: one learner writes scout code, another builds the room classifier.